Esse ai resolve com CCW

Thomaz Miranda, Miguel Batista, Joao Arthur Marques

February 27, 2026

Contents

1 Data Structures 1
- 1.1 Fenwick Tree (Binary Indexed Tree) 1
- 1.2 Segment Tree (Range Query + Point Update) 1
- 1.3 Segment Tree (Range Query + Range Update) 1
- 1.4 Sparse Table (Idempotent Range Query) 2
- 1.5 Disjoint Set Union (Union-Find) 2
- 1.6 Implicit Treap (Sequence Treap) 2
- 1.7 Order-Statistic Tree (PBDS) 2
- 1.8 Convex Hull Trick Dinamico 2

2 Graphs 3
- 2.1 Dijkstra's Shortest Paths 3
- 2.2 Dinic's Maximum Flow (with Scaling) 3
- 2.3 Min Cost Max Flow (MCMF) 3
- 2.4 Bridge Detection (Tarjan) 4
- 2.5 Strongly Connected Components (Kosaraju) 4
- 2.6 2-SAT 4
- 2.7 Topological Sort (Kahn's Algorithm) 5
- 2.8 LCA com binary lifting 5
- 2.9 Heavy Light Decomposition (Path query + Path update) 5
- 2.10 Centroid Decomposition - pares a distancia k 5

3 Math 6
- 3.1 Modular Integer 6
- 3.2 Modular Arithmetic Helpers 6
- 3.3 Linear Sieve (prime precomputation) 6
- 3.4 FFT/NTT Convolution 6
- 3.5 Matrix template 7
- 3.6 Gauss 7

4 Strings 7
- 4.1 Knuth-Morris-Pratt (KMP) 7
- 4.2 Aho-Corasick Automaton 8
- 4.3 Suffix Array - O(n log n) 8

5 Dynamic Programming 8
- 5.1 Divide and Conquer DP Optimization 8
- 5.2 Sum over Subsets DP (SOS DP) 8

6 Geometry 8
- 6.1 Integer Geometry Primitives 8
- 6.2 geo_double 9
- 6.3 Convex Hull (Monotone Chain) 9
- 6.4 halfplane 10

7 Extra 10
- 7.1 template 10
- 7.2 Pragmas 10

1 Data Structures

1.1 Fenwick Tree (Binary Indexed Tree) 1e1

Supports point updates and prefix/range sum queries in logarithmic time using a 1-indexed BIT.

Complexity: O(log N) per op, O(N)

```
8eb struct Bit {
4de     ll n;
06c     v64 bit;
dd0     Bit(ll _n = 0) : n(_n), bit(n + 1) {}
84e     Bit(v64& v) : n(sz(v)), bit(n + 1) {
518         for (ll i = 1; i <= n; i++) {
671             bit[i] += v[i - 1];
c8f             ll j = i + (i & -i);
b8a             if (j <= n) bit[j] += bit[i];
154         }
3f6     }
e55     void update(ll i, ll x) { // soma x na posicao i
b64         for (i++; i <= n; i += i & -i) bit[i] += x;
6f4     }
2c0     ll pref(ll i) { // soma [0, i]
b73         ll ret = 0;
4d3         for (i++; i; i -= i & -i) ret += bit[i];
edf         return ret;
af2     }
235     ll query(ll l, ll r) {  // soma [l, r]
89b         return pref(r) - pref(l - 1);
aa0     }
f46     ll upper_bound(ll x) {
62d         ll p = 0;
370         for (ll i = __lg(n); i+1; i--)
6f5             if (p + (1<<i) <= n and bit[p + (1<<i)] <= x)
68e                 x -= bit[p += (1 << i)];
74e         return p;
3d3     }
1e1 };
```

1.2 Segment Tree (Range Query + Point Update) bd2

supports point updates and range queries.

Complexity: O(log N) per op, O(N)

```
3c9 struct node {
f96     ll val = 0;

22b     static node comb(const node& a, const node& b) {
6c8         return {a.val + b.val};
ca4     }
d99 };

7bc template<typename T> struct segtree {
4de   ll n;
53f   T neutral;
a48   vector<T> tree;

c41   segtree<T>(ll _n, T _neutral = T()) {
7b9     n = _n, neutral = _neutral;
aa3     tree.resize(2*n+1, neutral);
40f   }

dc3   void set_leaves(vector<T> &leaves) {
4f7     copy(leaves.begin(), leaves.end(), tree.begin() + n);

6e6     for (ll i = n - 1; i > 0; i--) tree[i] = T::comb(tree[2 * i],
 tree[2 * i + 1]);
144   }

418   void update(ll i, T v) {
83b     i += n;
11e     tree[i] = v;
b8f     while (i > 1) {
011       i /= 2;
00e       tree[i] = T::comb(tree[2 * i], tree[2 * i + 1]);
13c     }
cec   }

0fc   T query(ll i, ll j) {
3d8     T rl = neutral, rr = neutral;
75a     for(i += n, j += n; i <= j; i /= 2, j /= 2){
bf3       if((i&1) == 1) rl = T::comb(rl, tree[i++]);
ebf       if((j&1) == 0) rr = T::comb(tree[j--], rr);
2ec     }
814     return T::comb(rl, rr);
600   }
dfd };
```

1.3 Segment Tree (Range Query + Range Update) 9ab

Tree for range queries with a customizable combine; supports range updates and range queries.

Complexity: O(log N) per op, O(N) to build

```
bf2 struct Node {
325     ll sum, cnt;
298     Node operator*(const Node &o) const { return {sum + o.sum,
cnt + o.cnt}; }
81b };

820 struct Update {
8bb     ll add = 0;
c81     optional<ll> set;

e0b     Node operator()(const Node &n) const {
49b         ll res = set.has_value() ? *set * n.cnt : n.sum;
9dd         return {res + add * n.cnt, n.cnt};
bf2     }

487     Update operator+(const Update &o) const {
ae9         Update res = *this;
035         if (o.set.has_value()) {
5ad             res.set = o.set;
d0d             res.add = 0;
902         }
7fc         if (o.add != 0) {
354             if (res.set.has_value()) *res.set += o.add;
ade             else res.add += o.add;
818         }
b50         return res;
b1d     }
d61 };

188 template<typename T, typename U> struct segtree {
0b8   ll s, h;

d0f   T id;
226   vector<T> val;

11c   U noop;
4af   vector<bool> dirty;
578   vector<U> prop;

118   explicit segtree(ll ts, T tid = T(), U tnoop = U()) {
180     id = tid, noop = tnoop;
85c     for (s = 1, h = 1; s < ts; ) s *= 2, h++;

173     val.assign(2*s, id);
fa8     dirty.assign(2*s, false);
b4f     prop.assign(2*s, noop);
c3b   }

cb1   void set_leaves(vector<T> &lvs) {
e2d     copy(lvs.begin(), lvs.end(), val.begin() + s);
```

```
8b0     for (ll i = s - 1; i > 0; i--) val[i] = val[2 * i] * val[2 *
 i + 1];
fa8     dirty.assign(2*s, false);
b4f     prop.assign(2*s, noop);
d52   }

732   void apply(ll i, U &upd) {
0ef     val[i] = upd(val[i]);
a35     if(i < s) {
083       prop[i] = prop[i] + upd;
deb       dirty[i] = true;
9df     }
af7   }

ae6   void pull(ll i) {
33b     for (ll l = i/2; l; l /= 2) {
218       T comb = val[2*l] * val[2*l+1];
5ba       val[l] = prop[l](comb);
e5c     }
aa8   }

cb0   void push(ll i) {
b93     for (ll h = h; h > 0; h--) {
b64       ll l = i >> h;

2fa       if (dirty[l]) {
02d         apply(2*l, prop[l]);
aed         apply(2*l+1, prop[l]);

62d         prop[l] = noop;
f99         dirty[l] = false;
ed9       }
613     }
b70   }

98b   void update(ll i, ll j, U upd) {
5ba     i += s, j += s;
f44     push(i), push(j);

689     for (ll l = i, r = j; l <= r; l /= 2, r /= 2) {
b7e       if((l&1) == 1) apply(l++, upd);
b0a       if((r&1) == 0) apply(r--, upd);
30c     }

999     pull(i), pull(j);
fda   }

0fc   T query(ll i, ll j){
5ba     i += s, j += s;
f44     push(i), push(j);

494     T rl = id, rr = id;
a82     for(; i <= j; i /= 2, j /= 2){
a29       if((i&1) == 1) rl = rl * val[i++];
3dd       if((j&1) == 0) rr = val[j--] * rr;
e07     }
f7e     return rl * rr;
62e   }
f31 };
```

1.4 Sparse Table (Idempotent Range Query) 91e

Preprocesses static array to answer idempotent range queries (e.g., min/max) in O(1) after O(N log N) build.

Complexity: O(N log N) build, O(1) query; O(N log N) space

```
991 struct sparse {
044     vector<v64> m;

bd3     sparse(v64& v) {
dce         ll n = sz(v);
eb5         ll logn = 64 - __builtin_clzll(n - 1);
a7d         m.resize(n+1, v64(logn+1));

7c9         forn(i, 0, n) m[i][0] = v[i];

aae         for (ll j = 1; (1 << j) <= n; j++)
82d         for (ll i = 0; i + (1 << j) <= n; i++)
631             m[i][j] = min(m[i][j-1], m[i + (1 << (j-1))][j-1]);
214     }

4de     ll query(ll a, ll b) {
b44         ll j = __builtin_clzll(1) - __builtin_clzll(b - a + 1);
a6f         return min(m[a][j], m[b - (1 << j) + 1][j]);
992     }
91e };
```

1.5 Disjoint Set Union (Union-Find) e67

Supports find with path compression and union by size to maintain dynamic connectivity of disjoint sets.

Complexity: O(alpha(N)) amortized per op, O(N)

```
8d3 struct dsu {
315     v64 id, len;

947
    dsu(ll n) : id(n), len(n, 1) { iota(id.begin(), id.end(), 0); }

f21
    ll find(ll a) { return a == id[a] ? a : id[a] = find(id[a]); }

2fe     bool uni(ll a, ll b) {
605         a = find(a), b = find(b);
5c2         if (a == b) return false;
8d6         if (len[a] < len[b]) swap(a, b);
f5f         len[a] += len[b], id[b] = a;
8a6         return true;
9b2     }
e67 };
```

1.6 Implicit Treap (Sequence Treap) 2a5

Maintains a sequence with split and merge operations using randomized priorities and subtree sizes.

Complexity: O(log N) expected per op, O(N)

```
125 struct Treap{
348     ll val;
0ce     ll prio, size;
330     vector<Treap*> kids;
b02     Treap(ll c): val(c), prio(rand()), size(1),
680         kids({NULL,NULL}){};
494 };

464 ll size(Treap *me){return me ? me->size : 0;}

e86 void rsz(Treap* me){me -> size =
24d     1 + size(me->kids[0]) + size(me->kids[1]);}

e8f vector<Treap*> split(Treap *me, ll idx){
878     if(!me) return {NULL,NULL};
032     vector<Treap*> out;

52a     if(size(me->kids[0]) < idx){
e1c         auto aux = split(me->kids[1],
312             idx - size(me->kids[0]) -1);
409         me->kids[1] = aux[0];
b14         rsz(me);
abb         out = {me, aux[1]};
aaa     }else{
c8a         auto aux = split(me->kids[0], idx);
c89         me->kids[0] = aux[1];
b14         rsz(me);
3cb         out = {aux[0], me};
d61     }
fe8     return out;
e7d }

b85 Treap* merge(Treap *left, Treap *right){
c10     if(left == NULL) return right;
096     if(right == NULL) return left;

671     Treap* out;

38d     if(left->prio < right->prio){
d90         left->kids[1] = merge(left->kids[1], right);
122         rsz(left);
d7a         out = left;
bbb     }else{
cea         right->kids[0] = merge(left, right->kids[0]);
e85         rsz(right);
015         out = right;
2f1     }
fe8     return out;
499 }
```

1.7 Order-Statistic Tree (PBDS) 9a7

Wraps __gnu_pbds tree to support order_of_key and find_by_order operations on a sorted set.

Complexity: O(log N) per op, O(N)

```
774 #include <ext/pb_ds/assoc_container.hpp>
30f #include <ext/pb_ds/tree_policy.hpp>

0d7 using namespace __gnu_pbds;

63c #define ordered_set tree<p64, null_type,less<p64>, rb_tree_tag,
tree_order_statistics_node_update>

e8d int main() {
7bf     ordered_set s;
d92     s.find_by_order(position);
d91     s.order_of_key(value);
a48 }
```

1.8 Convex Hull Trick Dinamico 60c

para double, use INF = 1/.0, div(a, b) = a/b update(x) atualiza o ponto de intersecao da reta x overlap(x) verifica se a reta x sobrepoe a proxima add(a, b) adiciona reta

da forma ax + b query(x) computa maximo de ax + b para entre as retas O(log(n)) amortizado por insercao O(log(n)) por query

```
72c struct Line {
073     mutable ll a, b, p;
8e3     bool operator<(const Line& o) const { return a < o.a; }
abf     bool operator<(ll x) const { return p < x; }
469 };

1b7 struct CHT : multiset<Line, less<>> {
33a     ll div(ll a, ll b) {
a20         return a / b - ((a ^ b) < 0 and a % b);
a8a     }

bbb     void update(iterator x) {
459         if (next(x) == end()) x->p = INF;
eec         else if (x->a == next(x)->a) x->p = x->b >= next(x)->b ?
 INF : -INF;
424         else x->p = div(next(x)->b - x->b, x->a - next(x)->a);
d37     }

71c     bool overlap(iterator x) {
f18         update(x);
cfa         if (next(x) == end()) return 0;
a4a         if (x->a == next(x)->a) return x->b >= next(x)->b;
d40         return x->p >= next(x)->p;
901     }

176     void add(ll a, ll b) {
1c7         auto x = insert({a, b, 0});
4ab         while (overlap(x)) erase(next(x)), update(x);
dbc         if (x != begin() and !overlap(prev(x))) x = prev(x),
update(x);
0fc         while (x != begin() and overlap(prev(x)))
4d2             x = prev(x), erase(next(x)), update(x);
48f     }

4ad     ll query(ll x) {
229         assert(!empty());
7d1         auto l = *lower_bound(x);
d41 #warning cuidado com overflow!
aba         return l.a * x + l.b;
3f5     }
f1f };
```

2 Graphs

2.1 Dijkstra's Shortest Paths 923

Computes single-source shortest paths on non-negative weighted graphs using a priority queue.

Complexity: O((N + M) log N), O(N + M)

```
c6d vector<vector<p64>> g;

// d = distance | p = from/path
ff3 void dijkstra(ll s, v64 &d, v64& p) {
5b8     ll n = sz(g);
355     d.assign(n, INF);
d8d     p.assign(n, -1);

d66     d[s] = 0;
930     priority_queue<p64> pq;
7ba     pq.push({0, s});
502     while (!pq.empty()) {
5cd         ll u = pq.top().second;
6fd         ll d_u = -pq.top().first;
716         pq.pop();

211         if (d_u != d[u]) continue;

bf7         for (auto edge : g[u]) {
615             ll v = edge.first;
61d             ll w_v = edge.second;

f35             if (d[u] + w_v < d[v]) {
7ca                 d[v] = d[u] + w_v;
e42                 p[v] = u;
2a6                 pq.push({-d[v], v});
e72             }
138         }
461     }
2f7 }
```

2.2 Dinic's Maximum Flow (with Scaling) 30a

Computes max flow using Dinic's algorithm with optional capacity scaling to speed up BFS levels.

Complexity: O(E V$\hat{}${}2) worst-case, O(E)

```
472 struct dinitz {
d76     const bool scaling = true;
d74     ll lim;
670     struct edge {
283         ll to, cap, rev, flow;
7f9         bool res;
764         edge(ll to_, ll cap_, ll rev_, bool res_)
a94
            : to(to_), cap(cap_), rev(rev_), flow(0), res(res_) {}
eb4     };

002     vector<vector<edge>> g;
f39     v64 lev, beg;
a71     ll F;
17f     dinitz(ll n) : g(n), F(0) {}

f3e     void add(ll a, ll b, ll c) {
de2         g[a].emplace_back(b, c, sz(g[b]), false);
a1d         g[b].emplace_back(a, 0, sz(g[a])-1, true);
3d3     }

6d8     bool bfs(ll s, ll t) {
2b7         lev = v64(sz(g), -1); lev[s] = 0;
71b         beg = v64(sz(g), 0);
7a6         queue<ll> q; q.push(s);
647         while (sz(q)) {
c79             ll u = q.front(); q.pop();
bd9             for (auto& i : g[u]) {
e4f
                if (lev[i.to] != -1 || (i.flow == i.cap)) continue;
17e                 if (scaling && i.cap - i.flow < lim) continue;
185                 lev[i.to] = lev[u] + 1;
8ca                 q.push(i.to);
00d             }
823         }
0de         return lev[t] != -1;
68b     }
bae     ll dfs(ll v, ll s, ll f = INF) {
47d         if (!f || v == s) return f;
78b         for (ll& i = beg[v]; i < sz(g[v]); i++) {
027             auto& e = g[v][i];
206             if (lev[e.to] != lev[v] + 1) continue;
a30             ll foi = dfs(e.to, s, min(f, e.cap - e.flow));
749             if (!foi) continue;
3c5             e.flow += foi, g[e.to][e.rev].flow -= foi;
45c             return foi;
387         }
bb3         return 0;
f09     }

074     ll max_flow(ll s, ll t) {
a86         for (lim = scaling ? (1<<30) : 1; lim; lim /= 2)
69c             while (bfs(s, t)) while (ll ff = dfs(s, t)) F += ff;
4ff         return F;
370     }

e30     void reset() {
59f         F = 0;
843         for (auto& edges : g) for (auto& e : edges) e.flow = 0;
5d0     }
30a };
```

2.3 Min Cost Max Flow (MCMF) 0a9

min_cost_flow(s, t, f) computes the pair (flow, cost) where max(flow) <= f with min(cost). min_cost_flow(s, t) -> Maximum flow with minimum cost from s to t. For DAGs, SPFA can be replaced with DP to avoid O(nm) initial cost. If no negative cost edges exist, SPFA is not needed.

Complexity: O(nm + f * m log n), O(n + m)

```
123 template<typename T> struct mcmf {
670     struct edge {
435         ll to, rev, flow, cap; // destination, reverse edge id,
current flow, capacity
7f9         bool res; // is reverse edge
635         T cost; // cost per unit of flow
892         edge() : to(0), rev(0), flow(0), cap(0), cost(0), res(
false) {}
92e
        edge(ll to_, ll rev_, ll flow_, ll cap_, T cost_, bool res_)
f8d             : to(to_), rev(rev_), flow(flow_), cap(cap_), res(
res_), cost(cost_) {}
2cb     };

002     vector<vector<edge>> g;
ade     v64 par_idx, par;
f1e     T inf;
a03     vector<T> dist;

673     mcmf(ll n) : g(n), par_idx(n), par(n), inf(numeric_limits<T
>::max()/3) {}

6dd     void add(ll u, ll v, ll w, T cost) { // edge from u to v
with capacity w and cost
efb         edge a = edge(v, sz(g[v]), 0, w, cost, false);
32a         edge b = edge(u, sz(g[u]), 0, 0, -cost, true);

b24         g[u].push_back(a);
c12         g[v].push_back(b);
fc8     }
```

```
b27
    vector<T> spfa(ll s) { // not needed if no negative cost edges
ed2         deque<ll> q;
9f3         vector<bool> is_inside(sz(g), 0);
d36         dist = vector<T>(sz(g), inf);

a93         dist[s] = 0;
a30         q.push_back(s);
ecb         is_inside[s] = true;

14d         while (!q.empty()) {
017             ll v = q.front();
ced             q.pop_front();
48d             is_inside[v] = false;

2ec             forn(i,0,sz(g[v])){
9d4                 auto [to, rev, flow, cap, res, cost] = g[v][i];
e61                 if (flow < cap and dist[v] + cost < dist[to]) {
943                     dist[to] = dist[v] + cost;

ed6                     if (is_inside[to]) continue;
020                     if (!q.empty() and dist[to] > dist[q.front()
]) q.push_back(to);
b33                     else q.push_front(to);
b52                     is_inside[to] = true;
2d1                 }
a8e             }
9e6         }
8d7         return dist;
79c     }
9aa     bool dijkstra(ll s, ll t, vector<T>& pot) {
90e         priority_queue<pair<T, ll>, vector<pair<T, ll>>, greater
<>> q;
d36         dist = vector<T>(sz(g), inf);
a93         dist[s] = 0;
115         q.emplace(0, s);
647         while (sz(q)) {
91b             auto [d, v] = q.top();
833             q.pop();
68b             if (dist[v] < d) continue;
2ec             forn(i,0,sz(g[v])) {
9d4                 auto [to, rev, flow, cap, res, cost] = g[v][i];
e8c                 cost += pot[v] - pot[to];
e61                 if (flow < cap and dist[v] + cost < dist[to]) {
943                     dist[to] = dist[v] + cost;
441                     q.emplace(dist[to], to);
88b                     par_idx[to] = i, par[to] = v;
873                 }
51f             }
5a5         }
1d4         return dist[t] < inf;
75e     }

928     pair<ll, T> min_cost_flow(ll s, ll t, ll flow = INF) {
437         vector<T> pot(sz(g), 0);
9e4         pot = spfa(s); // change shortest path algorithm here
         // no negative costs: nothing needed
         // DAG: use DP instead
c62         ll f = 0;
ce8         T ret = 0;
4a0         while (f < flow and dijkstra(s, t, pot)) {
945             forn(i,0,sz(g))
d2a                 if (dist[i] < inf) pot[i] += dist[i];
306             ll mn_flow = flow - f, u = t;
045             while (u != s){
90f                 mn_flow = min(mn_flow,
07d                     g[par[u]][par_idx[u]].cap - g[par[u]][
par_idx[u]].flow);
3d1                 u = par[u];
935             }

1f2             ret += pot[t] * mn_flow;

476             u = t;
045             while (u != s) {
e09                 g[par[u]][par_idx[u]].flow += mn_flow;
d98                 g[u][g[par[u]][par_idx[u]].rev].flow -= mn_flow;
3d1                 u = par[u];
bcc             }

04d             f += mn_flow;
957         }

15b         return make_pair(f, ret);
8e9     }

    // Optional: returns original edges where flow == cap
935     vector<p64> recover() {
e6a         vector<p64> used;
b25         forn(i, 0, sz(g)) for (edge e : g[i])
587
            if(e.flow == e.cap && !e.res) used.push_back({i, e.to});
f6b         return used;
f57     }
0a9 };
```

2.4 Bridge Detection (Tarjan) ecb

Finds all bridges in an undirected graph via DFS timestamps and low-link values.
Complexity: O(N + M), O(N + M)

```
a64 vector<v64> g;
591 vector<bool> visited;
023 vector<ll> tin, low;
ddf ll timer = 0;

081 void dfs(ll u, ll p = -1) {
2a9     visited[u] = true;
ae3     tin[u] = low[u] = timer++;
cd0     for (ll v : g[u]) {
730         if (v == p) continue;

d53         if (visited[v]) {
34f             low[u] = min(low[u], tin[v]);
caf         } else {
95e             dfs(v, u);
ab6             low[u] = min(low[u], low[v]);
975             if (low[v] > tin[u]) {
                 // THIS IS A BRIDGE
4b8             }
450         }
e83     }
7a4 }

822 void find_bridges() {
451     timer = 0;
411     visited.assign(n, false);
cfd     tin.assign(n, -1);
dc4     low.assign(n, -1);
522     forn(i, 0, n) {
b1c         if (!visited[i])
1e5             dfs(i);
bf3     }
bf3 }
```

2.5 Strongly Connected Components (Kosaraju) 14f

Computes SCCs using two DFS passes and builds the condensation graph.
Complexity: O(N + M), O(N + M)

```
591 vector<bool> visited;

297 void dfs(ll v, vector<v64>& g, vector<ll> &out) {
e75     visited[v] = true;
819     for(auto u : g[v]) if(!visited[u]) dfs(u, g, out);
3ad     out.push_back(v);
b7f }

64d vector<v64> scc(vector<v64>& g) {
625     int n = sz(g);
cb9     v64 order, roots(n, 0);

c44     vector<v64> adj_rev(n);
0c2     forn(u, 0, n) for (ll v : g[u]) adj_rev[v].push_back(u);

411     visited.assign(n, false);
2b4     forn(i, 0, n) if (!visited[i]) dfs(i, g, order);
b3a     reverse(order.begin(), order.end());

411     visited.assign(n, false);
de0     ll curr_comp = 0;
0ee     for (auto v : order) {
451         if (!visited[v]) {
a76             v64 component; dfs(v, adj_rev, component);
fe3             for (auto u : component) roots[u] = curr_comp;
5f2             curr_comp++;
19c         }
74d     }

e7f     set<p64> edges;
556     vector<v64> cond_g(curr_comp);
c6b     forn(u, 0, n) {
7b9         for (auto v : g[u]) {
2dd             if (roots[u] != roots[v] && !edges.count({roots[u],
roots[v]})) {
2b9                 cond_g[roots[u]].push_back(roots[v]);
893                 edges.emplace(roots[u], roots[v]);
fbe             }
f76         }
3b9     }
594     return cond_g;
22c }
```

2.6 2-SAT 3ee

solve() retorna um par, o first fala se eh possivel atribuir, o second fala se cada variavel eh verdadeira O(|V|+|E|) = O(#variaveis + #restricoes)

```
2b7 #include<bits/stdc++.h>
ca4 using namespace std;
```

```
ad1 typedef long long ll;
513 typedef vector<ll> v64;

138 struct sat {
446     ll n, tot;
a64     vector<v64> g;
d2a     v64 vis, comp, id, ans;
11f     stack<ll> s;

141     sat() {}
5f5     sat(ll n_) : n(n_), tot(n), g(2*n) {}

81c     ll dfs(ll i, ll& t) {
02f         ll lo = id[i] = t++;
efc         s.push(i), vis[i] = 2;
9c7         for (ll j : g[i]) {
740             if (!vis[j]) lo = min(lo, dfs(j, t));
994             else if (vis[j] == 2) lo = min(lo, id[j]);
3aa         }
3de         if (lo == id[i]) while (1) {
ad9             ll u = s.top(); s.pop();
9c5             vis[u] = 1, comp[u] = i;
91d
            if ((u>>1) < n and ans[u>>1] == -1) ans[u>>1] = ~u&1;
2ef             if (u == i) break;
a00         }
253         return lo;
04d     }

286     void add_impl(ll x, ll y) { // x -> y = !x ou y
26a         x = x >= 0 ? 2*x : -2*x-1;
2b8         y = y >= 0 ? 2*y : -2*y-1;
a1e         g[x].push_back(y);
1e2         g[y^1].push_back(x^1);
677     }
9b5     void add_cl(ll x, ll y) { // x ou y
0b5         add_impl(~x, y);
135     }
37f     void add_xor(ll x, ll y) { // x xor y
0b7         add_cl(x, y), add_cl(~x, ~y);
4ac     }
680     void add_eq(ll x, ll y) { // x = y
c86         add_xor(~x, y);
9b1     }
874     void add_true(ll x) { // x = T
18b         add_impl(~x, x);
ea8     }
2b7     void at_most_one(v64 v) { // no max um verdadeiro
051         g.resize(2*(tot+sz(v)));
758         for (ll i = 0; i < sz(v); i++) {
8c9             add_impl(tot+i, ~v[i]);
a8f             if (i) {
b6a                 add_impl(tot+i, tot+i-1);
3d3                 add_impl(v[i], tot+i-1);
0f7             }
691         }
253         tot += sz(v);
9fb     }

a55     pair<bool, v64> solve() {
6d0         ans = v64(n, -1);
b1b         ll t = 0;
86d         vis = comp = id = v64(2*tot, 0);
fe3         for (ll i = 0; i < 2*tot; i++) if (!vis[i]) dfs(i, t);
d99         for (ll i = 0; i < tot; i++)
4c9             if (comp[2*i] == comp[2*i+1]) return {false, {}};
997         return {true, ans};
d80     }
47d };
```

2.7 Topological Sort (Kahn's Algorithm) c8f

Produces a topological ordering of a DAG using indegree counting and a queue-like frontier.

Complexity: O(N + M), O(N)

```
1f7 v64 topo_sort(const vector<v64>& g) {
96b     v64 indeg(sz(g)), q;
edb     for (auto& li : g) for (int x : li) indeg[x]++;
707     forn(i, 0, sz(g)) if (indeg[i] == 0) q.push_back(i);
7c5     forn(j, 0, sz(q)) for(int x : g[q[j]]) if(--indeg[x] == 0) q.
push_back(x);
bef     return q;
c8f }
```

2.8 LCA com binary lifting 340

Assume que um vertice eh ancestral dele mesmo, ou seja, se a eh ancestral de b, lca(a, b) = a MAXLOG = ceil(log(MAX)) Complexidades: build - O(n log(n)) lca - O(log(n))

```
8a4 vector<v64> g(MAX);
883 ll n, p;
a8e ll parent[MAXLOG][MAX];
d7e ll in[MAX], out[MAX];

65b void dfs(ll k) {
fdf     in[k] = p++;
2e0     for(ll v: g[k]){
e58         if(in[v] == -1){
5ad             parent[0][v] = k;
6b4             dfs(v);
e1f         }
ca0     }
26f     out[k] = p++;
65e }

f50 void build(ll raiz) {
4c7     forn(i,0,n) parent[0][i] = i;
c63     p = 0, memset(in, -1, sizeof in);
ecb     dfs(raiz);
671     forn(k,1,MAXLOG) forn(i,0,n)
ed8         parent[k][i] = parent[k - 1][parent[k - 1][i]];
7ce }

64c bool anc(ll a, ll b) { // se a eh ancestral de b
bfe     return in[a] <= in[b] and out[a] >= out[b];
56c }

583 ll lca(ll a, ll b) {
86d     if (anc(a, b)) return a;
e52     if (anc(b, a)) return b;
fb3     for (ll k = MAXLOG - 1; k >= 0; k--)
499         if (!anc(parent[k][a], b)) a = parent[k][a];
793     return parent[0][a];
2ab }
```

2.9 Heavy Light Decomposition (Path query + Path update) 5d8

Decomposes a tree into vertex disjoint heavy paths and light edges such that the path from any leaf to the root contains at most log(n) light edges. All values initialized to the segtree default. Root must be 0. Uses a Lazy Segment Tree

Complexity: O((log N)$\hat{\{}$}2) per op, O(N) build

```
303 template <bool VALS_EDGES> struct HLD {
6fa     ll N, tim = 0;
e4f     vector<v64> adj;
f9b     v64 parent, sz, head, pos;
69d     vector<node> vseg;
cf3     std::unique_ptr<tree> seg;
d62     HLD(vector<v64> adj_, v64 vals)
d73         : N(sz(adj_)), adj(adj_), parent(N, -1), sz(N, 1),
e0d           head(N),pos(N),vseg(N, {0}){ dfsSz(0); dfsHld(0);
29f             seg = make_unique<tree>(0, N-1, vseg);
ff7         }
eb1     void dfsSz(ll v) { // get heavy son
d10         for (ll& u : adj[v]) {
44a             adj[u].erase(find(adj[u].begin(), adj[u].end(), v));
308             parent[u] = v;
a00             dfsSz(u);
246             sz[v] += sz[u];
4c3             if (sz[u] > sz[adj[v][0]]) swap(u, adj[v][0]);
22a         }
9be     }
7f7     void dfsHld(ll v) { // "linearizes" the tree
06c         pos[v] = tim++;
1c7         for (ll u : adj[v]) {
c0d             head[u] = (u == adj[v][0] ? head[v] : u);
9f9             dfsHld(u);
9af         }
e38     }
aa3     template <class B> void process(ll u, ll v, B op) {
2e7         for (;; v = parent[head[v]]) {
df0             if (pos[head[u]] > pos[head[v]]) swap(u, v);
a57             if (head[u] == head[v]) break;
ad9             op(pos[head[v]], pos[v]);
a34         }
bdc         if (pos[u] > pos[v]) swap(u, v);
adf         op(pos[u] + VALS_EDGES, pos[v]);
d5d     }
b5c     void modifyPath(ll u, ll v, ll val) {
3d8         process(u, v, [&](ll l, ll r) {
7ff             seg->range_update(l, r, {0, val}); // Modify
depending on problem
525         });
0da     }
960     ll queryPath(ll u, ll v) { // Modify depending on problem
567         ll res = -INF;
3d8         process(u, v, [&](ll l, ll r) {
083                 res = max(res, seg->query(l, r).val);
814         });
b50         return res;
2d1     }
a23     ll querySubtree(ll v) { // modifySubtree is similar
d9c         return seg->query(pos[v] + VALS_EDGES, pos[v] + sz[v] -
1).val;
226     }
5d8 };
```

2.10 Centroid Decomposition - pares a distancia k 9cb

Objetivo: contar #pares (u,v) com dist(u,v)=k na arvore. Ideia: em cada centroide c, para cada subarvore j: - path = profundidades d partindo de j (sem nos removidos) - somar cnt[k-d-1] ANTES de atualizar (evita dupla contagem entre subarvores) - depois, cnt[d+1]++ (a aresta (c,j) soma 1) Funcoes: dfs_sz (tamanhos), centroid (acha centroide), dfs (coleta depths), decomp (divide e combina). Complexidade: O(n log n) tempo; O(n) memoria. Cuidados: limites de cnt, k==0 trata a parte se quiser contar (u,u); rem[c] permanece marcado. escrito pelo gpt, cuidado

```
84c vector<ll> g[MAX];
d7c ll sz[MAX], rem[MAX];

b87 void dfs(v64& path, ll i, ll l=-1, ll d=0) {
547     path.push_back(d);
3d0
    for (ll j : g[i]) if (j != l and !rem[j]) dfs(path, j, i, d+1);
3e1 }

499 ll dfs_sz(ll i, ll l=-1) {
02c     sz[i] = 1;
05b
    for (ll j : g[i]) if (j != l and !rem[j]) sz[i] += dfs_sz(j, i);
191     return sz[i];
329 }

c46 ll centroid(ll i, ll l, ll size) {
51f
    for (ll j : g[i]) if (j != l and !rem[j] and sz[j] > size / 2)
735         return centroid(j, i, size);
d9a     return i;
c6f }

27a ll decomp(ll i, ll k) {
79c     ll c = centroid(i, i, dfs_sz(i));
a67     rem[c] = 1;

     // gasta O(n) aqui - dfs sem ir pros caras removidos
04b     ll ans = 0;
4eb     vector<ll> cnt(sz[i]);
878     cnt[0] = 1;
e65     for (ll j : g[c]) if (!rem[j]) {
04c         vector<ll> path;
baf         dfs(path, j);
392         for (ll d : path) if (0 <= k-d-1 and k-d-1 < sz[i])
285             ans += cnt[k-d-1];
477         for (ll d : path) cnt[d+1]++;
4d9     }

ffb     for (ll j : g[c]) if (!rem[j]) ans += decomp(j, k);
3f1     rem[c] = 0;
ba7     return ans;
595 }
```

3 Math

3.1 Modular Integer 76c

*Fixed-modulus integer type with +, -, *, /, and exponentiation; modulo should be prime for division via Fermat.*

Complexity: O(1) per arithmetic op (O(log E) for exponentiation), O(1)

```
a2f const ll MOD = 998244353;

f25 template<ll p> struct mod_int {
c68     ll expo(ll b, ll e) {
c85         ll ret = 1;
c87         while (e) {
cad             if (e % 2) ret = ret * b % p;
9d2             e /= 2, b = b * b % p;
c42         }
edf         return ret;
734     }
1f6     ll inv(ll b) { return expo(b, p-2); }

4d7     using m = mod_int;
aa3     ll v;
fe0     mod_int() : v(0) {}
e12     mod_int(ll v_) {
019         if (v_ >= p or v_ <= -p) v_ %= p;
bc6         if (v_ < 0) v_ += p;
2e7         v = v_;
7f3     }
74d     m& operator +=(const m& a) {
2fd         v += a.v;
ba5         if (v >= p) v -= p;
357         return *this;
c8b     }
eff     m& operator -=(const m& a) {
8b4         v -= a.v;
cc8         if (v < 0) v += p;
357         return *this;
f8d     }
4c4     m& operator *=(const m& a) {
686         v = v * a.v % p;
357         return *this;
c9b     }
3f9     m& operator /=(const m& a) {
5d6         v = v * inv(a.v) % p;
357         return *this;
62d     }
f98     m operator -() const { return m(-v); }
b3e     m& operator ^=(ll e) {
06d         if (e < 0) {
6e2             v = inv(v);
00c             e = -e;
275         }
284         v = expo(v, e);
         // possible optimization:
         // careful with 0^0
         // v = expo(v, e%(p-1));
357         return *this;
6ed     }
423     bool operator ==(const m& a) { return v == a.v; }
69f     bool operator !=(const m& a) { return v != a.v; }

1c6     friend istream& operator >>(istream& in, m& a) {
d1c         ll val; in >> val;
d48         a = m(val);
091         return in;
870     }
44f     friend ostream& operator <<(ostream& out, m a) {
5a0         return out << a.v;
214     }
399     friend m operator +(m a, m b) { return a += b; }
f9e     friend m operator -(m a, m b) { return a -= b; }
9c1     friend m operator *(m a, m b) { return a *= b; }
51b     friend m operator /(m a, m b) { return a /= b; }
08f     friend m operator ^(m a, ll e) { return a ^= e; }
20e };
0f4 typedef mod_int<MOD> mint;
```

3.2 Modular Arithmetic Helpers 012

Provides modular add/sub/mul, fast exponentiation, and modular inverse under fixed MOD.

Complexity: O(log E) for power/inverse

```
0f6 const ll MOD = 1_000_000_007;

d7e inline ll sum(ll a, ll b) { a += b; if (a >= MOD) a -= MOD;
return a; }
e0b inline ll sub(ll a, ll b) { a -= b; if (a < 0)  a += MOD; return
 a; }
d06 inline ll mult(ll a, ll b) { return (a * b) % MOD; }

f15 inline ll pot(ll base, ll exp) {
ce0     ll res = 1;
fb9     while (exp) {
3c3         if (exp & 1) res = mult(res, base);
ee9         base = mult(base, base);
ef0         exp >>= 1;
dcf     }
b50     return res;
24d }

840 inline ll inv_mod(ll a) {return pot(a, MOD-2);}
```

3.3 Linear Sieve (prime precomputation) a09

Computes primes up to N in O(N) using the smallest prime factor (lp). pr will contain all primes <= N. lp[x] stores the smallest prime dividing x. escrito pelo gpt, cuidado

Complexity: O(N)

```
01f const ll N = 10000000;
72d v64 lp(N+1), pr;

8e0 void sieve(){
506     forn(i, 2, N+1) {
719         if (lp[i] == 0) {
116             lp[i] = i;
bc0             pr.push_back(i);
d4f         }
6a8         for (ll j = 0; i * pr[j] <= N; ++j) {
a97             lp[i * pr[j]] = pr[j];
5f9             if (pr[j] == lp[i]) {
c2b                 break;
e08             }
20d         }
55a     }
910 }
```

3.4 FFT/NTT Convolution 02a

Implements iterative FFT over complex numbers and NTT over supported primes; provides convolution utility.

Complexity: O(N log N), O(N)

```
488 void get_roots(bool f, int n, vector<complex<double>>& roots) {
f26     const static double PI = acosl(-1);
71a     for (int i = 0; i < n/2; i++) {
b1e         double alpha = i*((2*PI)/n);
1a1         if (f) alpha = -alpha;
```

```
069         roots[i] = {cos(alpha), sin(alpha)};
804     }
de5 }

// Para NTT
9f7 template<int p>
97b void get_roots(bool f, int n, vector<mod_int<p>>& roots) {
1e6     mod_int<p> r;
de9     int ord;
57a     if (p == 998244353) {
9b6         r = 102292;
81b         ord = (1 << 23);
121     } else if (p == 754974721) {
43a         r = 739831874;
f0a         ord = (1 << 24);
d48     } else if (p == 167772161) {
a2a         r = 243;
033         ord = (1 << 25);
5a4     } else assert(false);

547     if (f) r = r^(p - 1 -ord/n);
ee2     else r = r^(ord/n);
be4     roots[0] = 1;
078     for (int i = 1; i < n/2; i++) roots[i] = roots[i-1]*r;
63f }

8a2 template<typename T> void fft(vector<T>& a, bool f, int N,
vector<int>& rev) {
bc7     for (int i = 0; i < N; i++) if (i < rev[i]) swap(a[i], a[rev
[i]]);
12b     int l, r, m;
cb4     vector<T> roots(N);
192     for (int n = 2; n <= N; n *= 2) {
0f4         get_roots(f, n, roots);
5dc         for (int pos = 0; pos < N; pos += n) {
432             l = pos + 0, r = pos + n/2, m = 0;
a88             while (m < n/2) {
297                 auto t = roots[m] * a[r];
254                 a[r] = a[l] - t;
b8f                 a[l] = a[l] + t;
2c9                 l++, r++, m++;
d89             }
1fd         }
185     }
235     if (f) {
1c5         auto invN = T(1) / T(N);
557         for (int i = 0; i < N; i++) a[i] = a[i] * invN;
256     }
1b1 }

bf5 template<typename T> vector<T> convolution(vector<T>& a, vector<
T>& b) {
87a     vector<T> l(a.begin(), a.end()), r(b.begin(), b.end());
a4b     int N = sz(l)+sz(r)-1;
f03     int n = 1, log_n = 0;
0a4     while (n <= N) n *= 2, log_n++;
808     vector<int> rev(n);
603     for (int i = 0; i < n; i++) {
434         rev[i] = 0;
f44         for (int j = 0; j < log_n; j++) if (i>>j&1)
4ff             rev[i] |= 1 << (log_n-1-j);
256     }
143     assert(N <= n);
fa4     l.resize(n);
7e4     r.resize(n);
56e     fft(l, false, n, rev);
fcf     fft(r, false, n, rev);
917     for (int i = 0; i < n; i++) l[i] *= r[i];
88b     fft(l, true, n, rev);
5e1     l.resize(N);
792     return l;
59c }

// NTT
74c template<int p, typename T>
b74 vector<mod_int<p>> ntt(vector<T>& a, vector<T>& b) {
d52
    vector<mod_int<p>> A(a.begin(), a.end()), B(b.begin(), b.end());
d29     return convolution(A, B);
543 }
```

3.5 Matrix template caa

- operator: multiplicacao (O(n*m*r)) opcional modo modular (#define MODULAR true) - operator^{}: exponenciacao rapida (O(log e)) - apply_transform(): aplica transformacao M^{}e * v*

```
945 #define MODULAR false
5ed template<typename T> struct matrix : vector<vector<T>> {
03e     ll n, m;
30f     void print() {
522         forn(i,0,n) {
064             forn(j,0,m) cout << (*this)[i][j] << " ";
1fb             cout << endl;
285         }
823     }

491     matrix(ll n_, ll m_, bool ident = false) :
b14
            vector<vector<T>>(n_, vector<T>(m_, 0)), n(n_), m(m_) {
94e         if (ident) {
df7             assert(n == m);
17a             forn(i,0,n) (*this)[i][i] = 1;
636         }
f73     }
b83     matrix(const vector<vector<T>>& c) : vector<vector<T>>(c),
98e         n(sz(c)), m(sz(c[0])) {}
efc     matrix(const initializer_list<initializer_list<T>>& c) {
f7e         vector<vector<T>> val;
212         for (auto& i : c) val.push_back(i);
303         *this = matrix(val);
c50     }

388     matrix<T> operator*(matrix<T>& r) {
1e2         assert(m == r.n);
82c         matrix<T> M(n, r.m);
630         forn(i,0,n) forn(k,0,m) forn(j, 0, r.m){
e34             T add = (*this)[i][k] * r[k][j];
f98 #if MODULAR
d41 #warning Usar matrix<ll> e soh colocar valores em [0, MOD) na
matriz!
8b6             M[i][j] += add%MOD;
983             if (M[i][j] >= MOD) M[i][j] -= MOD;
8c1 #else
7bb             M[i][j] += add;
f2e #endif
77d         }
474         return M;
ba1     }
528     matrix<T> operator^(ll e){
f10         matrix<T> M(n, n, true), at = *this;
c87         while (e) {
2e2             if (e&1) M = M*at;
cc2             e >>= 1;
c80             at = at*at;
eb6         }
474         return M;
ca3     }
582     void apply_transform(matrix M, ll e){
1c3         auto& v = *this;
c87         while (e) {
9ba             if (e&1) v = M*v;
cc2             e >>= 1;
419             M = M*M;
d86         }
4e5     }
f02 };
```

3.6 Gauss 406

*Resolve sistema linear Retornar um par com o numero de solucoes e alguma solucao, caso exista O(n^2 * m) escrito pelo gpt, cuidado*

```
67a template<typename T>
915 pair<ll, vector<T>> gauss(vector<vector<T>> a, vector<T> b) {
6ca     const double eps = 1e-6;
ee9     ll n = sz(a), m = sz(a[0]);
f4c     forn(i, 0, n) a[i].push_back(b[i]);

d6f     v64 where(m, -1);
d41     for (ll col = 0, row = 0; col < m && row < n; col++) {
b49         ll sel = row;
d96         forn(i, row, n)
e55             if(abs(a[i][col]) > abs(a[sel][col])) sel = i;
2c4         if(abs(a[sel][col]) < eps) continue;
552         forn(i, col, m+1)
dd2             swap(a[sel][i], a[row][i]);
2c3         where[col] = row;

959         forn(i, 0, n) if (i != row) {
96c             T c = a[i][col] / a[row][col];
b2d             forn(j, col, m+1)
c8f                 a[i][j] -= a[row][j] * c;
5a4         }
b70         row++;
11c     }

b1d     vector<T> ans(m, 0);
363     forn(i, 0, m) if (where[i] != -1)
12a         ans[i] = a[where[i]][m]/a[where[i]][i];
522     forn(i, 0, n) {
501         T sum = 0;
695         forn(j, 0, m)
5a9             sum += ans[j]*a[i][j];
b1f         if(abs(sum - a[i][m]) > eps)
6cd             return pair(0, vector<T>());
417     }

688     forn(i, 0, m) if (where[i] == -1)
018         return pair(INF, ans);
```

```
280     return pair(1, ans);
e8a }
```

4 Strings

4.1 Knuth-Morris-Pratt (KMP) 9d7

Computes prefix function and performs linear-time substring search with optional automaton construction.

Complexity: O(n + m), O(n)

```
a15 v64 pi(string& s) {
3ee     v64 p(sz(s));
3b9     for (ll i = 1, j = 0; i < sz(s); i++) {
a51         while (j and s[j] != s[i]) j = p[j-1];
973         if (s[j] == s[i]) j++;
f8c         p[i] = j;
215     }
74e     return p;
fc0 }

e89 v64 match(string& pat, string& s) {
cce     v64 p = pi(pat), match;
c0f     for (ll i = 0, j = 0; i < sz(s); i++) {
a5a         while (j and pat[j] != s[i]) j = p[j-1];
3e3         if (pat[j] == s[i]) j++;
1f4         if (j == sz(pat)) match.push_back(i-j+1), j = p[j-1];
c69     }
ed8     return match;
646 }

4a5 struct KMPaut : vector<v64> {
47c     KMPaut(){}
383     KMPaut (string& s) : vector<v64>(26, v64(sz(s)+1)) {
bb1         v64 p = pi(s);
04b         auto& aut = *this;
4fa         aut[s[0]-'a'][0] = 1;
19a         for (char c = 0; c < 26; c++)
906             for (int i = 1; i <= sz(s); i++)
42b
                aut[c][i] = s[i]-'a' == c ? i+1 : aut[c][p[i-1]];
e70     }
4be };
```

4.2 Aho-Corasick Automaton 8d8

Builds a trie with failure links for multi-pattern matching; insert is O(|s|), build is linear in total length, and queries run in linear time in the text.

Complexity: varies, O(total patterns length)

```
ea1 namespace aho {
05b     map<char, ll> to[MAX];
b0a     ll link[MAX], idx, term[MAX], exit[MAX], sobe[MAX];
5e1     vector<ll> max_match(MAX, 0);

bfc     void insert(string& s) {
4eb         ll at = 0;
b4f         for (char c : s) {
b68             auto it = to[at].find(c);
1c9             if (it == to[at].end()) at = to[at][c] = ++idx;
361             else at = it->second;
ff4         }
142         term[at]++, sobe[at]++;
6b7         max_match[at] = sz(s);
5f4     }

0a8     void build() {
848         queue<ll> q;
537         q.push(0);
dff         link[0] = exit[0] = -1;
647         while (sz(q)) {
aa7             ll i = q.front(); q.pop();
3c4             for (auto [c, j] : to[i]) {
7d8                 ll l = link[i];
102                 while (l != -1 and !to[l].count(c)) l = link[l];
7a5                 link[j] = l == -1 ? 0 : to[l][c];
3ab
                exit[j] = term[link[j]] ? link[j] : exit[link[j]];

058                 max_match[j] = max(max_match[link[j]], max_match
[j]);
6f2                 if (exit[j]+1) sobe[j] += sobe[exit[j]];
113                 q.push(j);
ed4             }
c0e         }
1ad     }
5e1     ll query(string& s) {
0db         ll at = 0, ans = 0;
b4f         for (char c : s){
1ca             while (at != -1 and !to[at].count(c)) at = link[at];
5b9             at = at == -1 ? 0 : to[at][c];
2b1             ans += sobe[at];
b85         }
ba7         return ans;
0bf     }
028     vector<ll> match_vec(string& s) {
c83         ll at = 0, n = sz(s);
d93         vector<ll> v(n, 0);
522         forn(i, 0, n){
827             char c = s[i];
1ca             while (at != -1 and !to[at].count(c)) at = link[at];
5b9             at = at == -1 ? 0 : to[at][c];

c84             v[i] = max_match[at]; // quero isso
5eb         }
6dc         return v;
f53     }
8d8 }
```

4.3 Suffix Array - O(n log n) f96

kasai recebe o suffix array e calcula lcp[i], o lcp entre s[sa[i],...,n-1] e s[sa[i+1],..,n-1] Complexidades: suffix_array - O(n log(n)) kasai - O(n)

```
ad7 v64 suffix_array(string s) {
59b     s.push_back('$'); // 0 caso v64 (CHECAR SE PODE)
14a     ll n = sz(s), N = max(n, 260ll);
b3e     v64 sa(n), ra(n);
828     forn(i, 0, n) sa[i] = i, ra[i] = s[i];

2e6     for(ll k = 0; k < n; k ? k *= 2 : k++) {
1db         v64 nsa(sa), nra(n), cnt(N);

fae         for(int i = 0; i < n; i++) nsa[i] = (nsa[i]-k+n)%n, cnt[
ra[i]]++;

e18         forn(i, 1, N) cnt[i] += cnt[i-1];
f62         for(ll i = n-1; i+1; i--) sa[--cnt[ra[nsa[i]]]] = nsa[i];

aec         for(ll i = 1, r = 0; i < n; i++) nra[sa[i]] = r += ra[sa
[i]] !=
f86             ra[sa[i-1]] or ra[(sa[i]+k)%n] != ra[(sa[i-1]+k)%n];
26b         ra = nra;
d5e         if (ra[sa[n-1]] == n-1) break;
02e     }
9f9     return v64(sa.begin()+1, sa.end());
7c3 }

c46 v64 kasai(string s, v64 sa) {
c30     ll n = sz(s), k = 0;
f7c     v64 ra(n), lcp(n);
540     forn(i, 0, n) ra[sa[i]] = i;

514     for (ll i = 0; i < n; i++, k -= !!k) {
199         if (ra[i] == n-1) { k = 0; continue; }
674         ll j = sa[ra[i]+1];
891         while (i+k < n and j+k < n and s[i+k] == s[j+k]) k++;
d98         lcp[ra[i]] = k;
b37     }
5ed     return lcp;
a47 }
```

5 Dynamic Programming

5.1 Divide and Conquer DP Optimization fb7

Optimizes DP transitions with quadrangle inequality/monge-like structure using divide-and-conquer over optimal decision points.

Complexity: O(K N log N) with O(1) cost, O(N)

```
f6c vector<v64> dp; // dp[n+1][2]

b8e void solve(ll k, ll l, ll r, ll lk, ll rk) {
de6     if (l > r) return;
f20     ll m = (l+r)/2, p = -1;
bff     auto& ans = dp[m][k&1] = INF;
f08     for (ll i = max(m, lk); i <= rk; i++) {
d73         ll at = dp[i+1][~k&1] + cost(m, i);
57d         if (at < ans) ans = at, p = i;
d63     }
1ee     solve(k, l, m-1, lk, p), solve(k, m+1, r, p, rk);
35b }

c5c ll dnc(ll n, ll k) {
321     dp[n][0] = dp[n][1] = 0;
390     forn(i,0,n) dp[i][0] = INF;
050     forn(i,1,k+1) solve(i, 0, n-i, 0, n-i);
8e7     return dp[0][k&1];
40f }
```

5.2 Sum over Subsets DP (SOS DP) 386

sos_dp_sub: for each mask, computes the sum of f[s] over all subsets s of mask (i.e., f'[mask] = sum of f[s] for s in mask). sos_dp_super: for each mask, computes the sum of f[s] over all supersets s of mask (i.e., f'[mask] = sum of f[s] for mask in s). input size must be a power of two (2^{}n).

Complexity: O(n * 2^{}n), O(2^{}n) mem

```
fb4 v64 sos_dp_sub(v64 f) {
f5d     ll n = __builtin_ctzll(sz(f));
310     assert((1ll<<n) == sz(f));
```

```
4cd     forn(i,0,n) forn(mask, 0, (1<<n))
796         if (mask>>i&1) f[mask] += f[mask^(1<<i)];
         // -= for inverse
abe     return f;
d55 }

074 v64 sos_dp_super(v64 f) {
f5d     ll n = __builtin_ctzll(sz(f));
310     assert((1ll<<n) == sz(f));

4cd     forn(i,0,n) forn(mask, 0, (1<<n))
a3c         if (~mask>>i&1) f[mask] += f[mask^(1<<i)];
         // -= for inverse
abe     return f;
e04 }
```

6 Geometry

6.1 Integer Geometry Primitives d44

Defines 2D point and line structures with orientation, area, and angle comparisons plus a sweep-line comparator.

```
b2a struct pt { // point
0be     ll x, y;
f6f     pt(ll x_ = 0, ll y_ = 0) : x(x_), y(y_) {}
5bc     bool operator < (const pt p) const {
95a         if (x != p.x) return x < p.x;
89c         return y < p.y;
dcd     }
a83     bool operator == (const pt p) const {
48a         return x == p.x && y == p.y;
277     }
cb9
    pt operator + (const pt p) const { return pt(x+p.x, y+p.y); }
a24
    pt operator - (const pt p) const { return pt(x-p.x, y-p.y); }
8f0     pt operator * (const ll c) const { return pt(x*c, y*c); }
60d     ll operator * (const pt p) const { return x*(ll)p.x + y*(ll)
p.y; } // dot product
d86     ll operator ^ (const pt p) const { return x*(ll)p.y - y*(ll)
p.x; } // cross product
5ed     friend istream& operator >> (istream& in, pt& p) {
e37         return in >> p.x >> p.y;
e45     }
1b5 };

b3a struct line { // line segment
730     pt p, q;

0d6     line() {}
4b8     line(pt p_, pt q_) : p(p_), q(q_) {}
7f9     bool operator < (const line l) const {
d1d         if (!(p == l.p)) return p < l.p;
d4a         return q < l.q;
2ca     }
e1c     bool operator == (const line l) const {
05e         return p == l.p && q == l.q;
b27     }
8d7     friend istream& operator >> (istream& in, line& r) {
4cb         return in >> r.p >> r.q;
858     }
ab7 };

5a2 ll sarea2(pt p, pt q, pt r) { // 2 * signed area
586     return (q-p)^(r-q);
bf4 }

e58 ll norm_sq(pt p){ return p*p;}

baa ll dist_sq(pt p, pt q){ return norm_sq(p - q);}

0cd bool ccw(pt p, pt q, pt r) { // true if p, q, r are
276     return sarea2(p, q, r) > 0;
42b }

c31 int quad(pt p) { // quadrant of a point
dbb     return (p.x<0)^3*(p.y<0);
fcf }

2df bool compare_angle(pt p, pt q) { // returns true if angle(p) <
angle(q)
9fc     if (quad(p) != quad(q)) return quad(p) < quad(q);
ea1     return ccw(q, pt(0, 0), p);
771 }

099 bool isinseg(pt p, line r) { // se p pertence ao seg de r
f65     pt a = r.p - p, b = r.q - p;
3e1     return (a ^ b) == 0 && (a * b) <= 0;
562 }

676 bool interseg(line r, line s) { // se o seg de r intersecta o
seg de s
19b     if (isinseg(r.p, s) or isinseg(r.q, s)
c21         or isinseg(s.p, r) or isinseg(s.q, r)) return 1;

13c     return ccw(r.p, r.q, s.p) != ccw(r.p, r.q, s.q) &&
413             ccw(s.p, s.q, r.p) != ccw(s.p, s.q, r.q);
bfb }

510 ll polarea(vector<pt> v) { // area do poligono
b73     ll ret = 0;
ced     forn(i, 0, sz(v))
382         ret += sarea2(pt(0, 0), v[i], v[(i + 1) % sz(v)]);
d03     return abs(ret);
76b }

// comparator for set to perform sweep line with segments
2c4 struct cmp_sweepline {
d80     bool operator () (const line& a, const line& b) const {
         // assumes segments have p < q
191         if (a.p == b.p) return ccw(a.p, a.q, b.q);
5bc         if (a.p.x != a.q.x && (b.p.x == b.q.x or a.p.x < b.p.x))
780             return ccw(a.p, a.q, b.p);
dc0         return ccw(a.p, b.q, b.p);
673     }
84a };
```

6.2 geo_double 736

```
088 typedef long double ld;

e3b const ld DINF = 1e18;
107 const ld eps = 1e-9;
4fd const ld pi = 2*acos(0);

b32 #define sq(x) ((x)*(x))

d00 ll cmp(ld x, ld y = 0, ld tol = eps) {
938     return (x <= y + tol) ? (x + tol < y) ? -1 : 0 : 1;
c8d }

b2a struct pt { // ponto
c1e     ld x, y;
3dd     pt(ld x_ = 0, ld y_ = 0) : x(x_), y(y_) {}
5bc     bool operator < (const pt p) const {
c4a         if (cmp(x, p.x) != 0) return cmp(x, p.x) < 0;
48b         if (cmp(y, p.y) != 0) return cmp(y, p.y) < 0;
bb3         return 0;
b2e     }
a83     bool operator == (const pt p) const {
0e5         return cmp(x, p.x) == 0 && cmp(y, p.y) == 0;
612     }
cb9
    pt operator + (const pt p) const { return pt(x+p.x, y+p.y); }
a24
    pt operator - (const pt p) const { return pt(x-p.x, y-p.y); }
4a8
    pt operator * (const ld c) const { return pt(x*c  , y*c  ); }
a60
    pt operator / (const ld c) const { return pt(x/c  , y/c  ); }
3b6     ld operator * (const pt p) const { return x*p.x + y*p.y; }
6df     ld operator ^ (const pt p) const { return x*p.y - y*p.x; }
5ed     friend istream& operator >> (istream& in, pt& p) {
e37         return in >> p.x >> p.y;
e45     }
6cb };

b3a struct line { // reta
730     pt p, q;
0d6     line() {}
4b8     line(pt p_, pt q_) : p(p_), q(q_) {}
8d7     friend istream& operator >> (istream& in, line& r) {
4cb         return in >> r.p >> r.q;
858     }
7ab };

8c5 ld norm_sq(pt p){ return p*p;}

b57 ld dist_sq(pt p, pt q){ return norm_sq(p - q);}

b47 ld dist(pt p,pt q){ return sqrt(dist_sq(p, q));}

589 ld angle(pt v) { // angulo do vetor com o eixo x
587     ld ang = atan2(v.y, v.x);
1e3     if (cmp(ang, 0) < 0) ang += 2*pi;
19c     return ang;
2e3 }

298 ld sarea(pt p, pt q, pt r) { // area com sinal
606     return ((q-p)^(r-q))/2;
1b1 }

e32 bool col(pt p, pt q, pt r) { // se p, q e r sao colin.
571     return cmp(sarea(p, q, r), 0) == 0;
e64 }

0cd bool ccw(pt p, pt q, pt r) { // se p, q, r sao ccw
2ba
```

```
    return cmp(sarea(p, q, r),0) > 0; // talvez vc queira = !!!!!!!
b7c }

13d ld polarea(vector<pt> v) { // area do poligono
9c5     ld ret = 0;
ced     forn(i, 0, sz(v))
6d4         ret += sarea(pt(0, 0), v[i], v[(i + 1) % sz(v)]);
d03     return abs(ret);
0a8 }

98d
ld get_t(pt v, line r) { // retorna t tal que t*v pertence a reta r
6ee     return (r.p^r.q) / ((r.p-r.q)^v);
a0a }

d5c pt inter(line r, line s) { // r inter s
9a6     if (cmp((r.p - r.q) ^ (s.p - s.q), 0) == 0) return pt(DINF,
DINF);
205     r.q = r.q - r.p, s.p = s.p - r.p, s.q = s.q - r.p;
543     return r.q * get_t(r.q, s) + r.p;
9be }
```

6.3 Convex Hull (Monotone Chain) 55e

Computes the convex hull of a set of points using Andrew's monotone chain; handles collinear points based on ccw condition.

Complexity: O(N log N), O(N)

```
eb2 vector<pt> convex_hull(vector<pt> &v) {
d76     v.erase(unique(v.begin(), v.end()), v.end());
c2b     if(sz(v) <=1) return v;
526     vector<pt> l, u;
6fe     forn(i, 0, sz(v)) {
e5c         while(sz(l) > 1 && !ccw(l.end()[-2], l.end()[-1], v[i]))
364             l.pop_back();
c35         l.push_back(v[i]);
d19     }
a62     for(ll i = sz(v)-1; i>=0; i--) {
ecb         while(sz(u) > 1 && !ccw(u.end()[-2], u.end()[-1], v[i]))
7a8             u.pop_back();
a95         u.push_back(v[i]);
d9f     }
364     l.pop_back();
7a8     u.pop_back();
82b     for(pt i : u) l.push_back(i);
792     return l;
55e }
```

6.4 halfplane 667

```
c79 mt19937_64 rng((ll) chrono::steady_clock::now().time_since_epoch
().count());

// só entra retas nao verticais, processar as retas verticais antes
e encodar essa info no minx e maxx
// se quiser o ponto mais acima, fazer best_dir = (0,1)
// dependendo do best_dir, best = (+-INF, +-INF)

877 pt hpi_1d(vector<line> v, pt best_dir, ll n, line suport, ll
minx, ll maxx){
3ec     pt low = inter(line(pt(minx, 0), pt(minx,1)), suport), upp =
 inter(line(pt(maxx, 0), pt(maxx,1)), suport);
522     forn(i, 0, n){
beb         pt intersection = inter(v[i], suport);
f1c         debug(low);
dda         debug(upp);
f9d         debug(intersection);
138         debug(v[i]);

5b8         if(!ccw(v[i].p, v[i].q, low)) low = intersection;
030         else if(!ccw(v[i].p, v[i].q, upp)) upp = intersection;
9b8     }

a7b     if(cmp(low.x, upp.x) > 0) return {DINF, DINF};

5b1     if(cmp(best_dir*low, best_dir*upp) <= 0) return upp;
b2b     return low;
b43 }

219 pt hpi(vector<line> v, pt best_dir, ll minx, ll maxx){
f21     shuffle(v.begin(), v.end(), rng);

dce     ll n = sz(v);
3a3     pt best(1, 1);

522     forn(i, 0, n){
20a         debug(best);
138         debug(v[i]);
121         if(ccw(v[i].p, v[i].q, best)) continue;
186         best = hpi_1d(v, best_dir, i, v[i], minx, maxx);
ff8         assert(!(best ==pt(DINF, DINF)));
4c5     }

f26     return best;
f0a }
```

7 Extra

7.1 template 8bc

```
2b7 #include <bits/stdc++.h>
ca4 using namespace std;

ad1 typedef long long ll;
482 typedef pair<ll, ll> p64;
513 typedef vector<ll> v64;

4d1 #define forn(i, s, e) for (ll i = (s); i < (e); i++)
66d #define sz(u) ((ll) u.size())
cb9 #define ln "\n"

ba0 #ifdef DEBUG
2d9 #define trace(u) u
931 #define _
8c1 #else
56a #define trace(u)
aef #define _ ios::sync_with_stdio(0); cin.tie(0)
f2e #endif

721 #define debug(u) trace(cout << #u " = " << u << ln)
e98 #define debugv(v) trace(cout << #v ": "; for (auto xx : v) cout
<< xx << " "; cout << ln)

563 const ll INF = 0x3f3f3f3f3f3f3f3fll;

e8d int main() {
fb1     _;
bb3     return 0;
0d7 }
```

7.2 Pragmas 29c

```
b4f #pragma GCC optimize("O3,unroll-loops,fast-math")
c07 #pragma GCC optimize("Ofast,fast-math,unroll-loops,inline")
89a #pragma GCC optimize("Os")
```